Desafío 1: Mesa de Pedidos Inteligente

Arquitectura Determinística con LangGraph, RAG Híbrido y Human-in-the-Loop

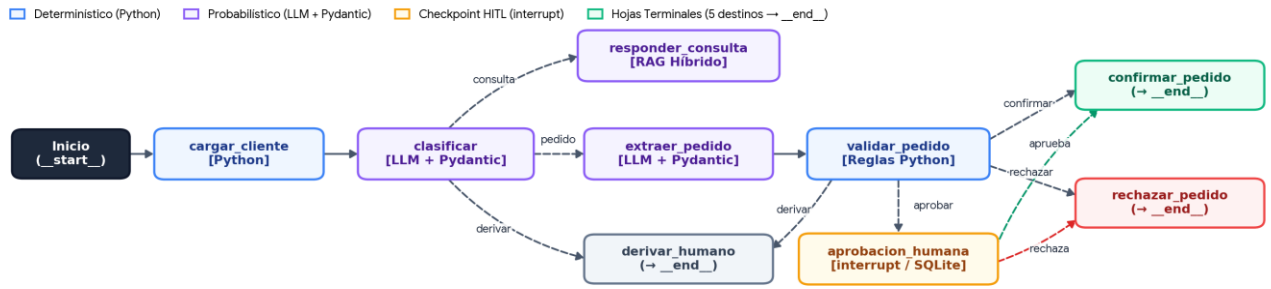
CASO: FERRETERIA INDUSTRIAL ANDINA – Modelo: Gemini Flash

1. Introducción y Planteo del Problema de Negocio

El procesamiento de mensajes en Ferretería Industrial Andina (WhatsApp y correo) exige conciliar lenguaje natural con rigor transaccional mayorista. Frente al antipatrón de delegar cálculos al LLM —vulnerable a alucinaciones aritméticas, no determinismo y saturación de contexto—, adoptamos un enfoque de IA Engineering: restringimos el modelo a tareas lingüísticas (clasificación, extracción y citas), resolviendo reglas comerciales, crédito y cómputos en Python puro determinístico.

2. Arquitectura del Grafo de Orquestación (LangGraph)

El sistema modela el ciclo de vida de cada mensaje mediante una máquina de estados finita en LangGraph, estructurada en 9 nodos, 3 routers condicionales y 5 hojas terminales:



Topología, Routers y Gestión de Estado (EstadoPedido)

- Routers como Pasamanos: No ejecutan lógica en las aristas; leen decisiones resueltas en el estado (intencion, validacion.ruta, decision_humana.tipo) y despachan al nodo siguiente, asegurando auditabilidad.
- Estado Tipado y Trazabilidad: EstadoPedido (TypedDict) usa un reducer acumulativo (Annotated[list[str], add]) sobre el campo log, generando una bitácora inmutable de transiciones para auditoría forense.
- Aislamiento de Capas: Los nodos LLM operan con esquemas Pydantic a temperatura 0.0; mientras que cargar_cliente y validar_pedido corren como funciones puras en Python sobre CSVs locales.

3. Estrategia RAG y Citación Verificable

El módulo normativo consulta cuatro documentos corporativos (Condiciones Comerciales, Devoluciones, Logística y Fichas Técnicas) bajo dos principios:

- Chunking Semántico Estructural: Se descartó el corte recursivo por caracteres. Un segmentador por expresiones regulares conserva cada artículo o ficha íntegro con sus metadatos de archivo, página y sección.
- Citación Anti-Alucinación: El LLM recibe fragmentos numerados [1], [2]... y devuelve solo los índices usados en Pydantic. Python traduce estos identificadores a metadatos reales (documento y página), impidiendo fuentes inventadas.

4. Persistencia Transaccional y Human-in-the-Loop (HITL)

- Para salvaguardar la rentabilidad, los pedidos que superan el saldo crediticio, presentan mora superior a 30 días o solicitan descuentos extraordinarios se interceptan en aprobacion_humana mediante la primitiva interrupt() de LangGraph.
- Durabilidad ante Fallos: El estado se serializa en disco con SqliteSaver (*checkpoints.sqlite*), asociando cada mensaje a un thread_id aislado. Si el servidor se apaga, la operación queda congelada sin pérdida de información.
- Payload de Decisión: Al pausarse, el nodo genera un resumen financiero consolidado (saldo, límite, mora, alertas y renglones valorizados con IVA) para que el supervisor decida sin consultar sistemas externos.
- Reanudación Asincrónica: El supervisor destraba el flujo mediante Command(resume="aprobar") o Command(resume="rechazar"), reanudando la ejecución directamente en confirmar_pedido o rechazar_pedido.

5. Evaluación Experimental y Métricas Globales (Golden Set)

La validación del sistema se efectuó de manera exhaustiva contra el conjunto formal de 20 casos de prueba, incorporando pausas de *throttling* (2.0s) para asegurar cumplimiento riguroso de las cuotas de la API gratuita.

Métrica de Evaluación	Muestra / Universo	Resultado Obtenido	Rendimiento (%)
Ruteo Correcto a Hoja Terminal	20 / 20 casos	20 aciertos	**100.0%**
Presencia de Dato Clave en Consultas	8 / 8 consultas	8 respuestas exactas	**100.0%**
Clasificación de Intención	20 / 20 casos	20 correctas	**100.0%**
Cita Documental Verificable	8 / 8 consultas	8 fuentes válidas	**100.0%**
Precisión en Extracción de Ítems	12 pedidos procesados	TP / (TP + FP)	**1.000**
Recall en Extracción de Ítems	12 pedidos procesados	TP / (TP + FN)	**1.000**

Nota metodológica: El 100% en dato clave valida el benchmark sintético, pero la auditoría cualitativa reveló discrepancias normativas sutiles (ver sección 6). Hojas alcanzadas: responder_consulta (8), confirmar_pedido (4), aprobacion_humana (4), rechazar_pedido (1) y derivar_humano (3).

6. Análisis Crítico de Casos Borde y Aprendizajes de Ingeniería

- Ceguera Vectorial y Retriever Híbrido (MSG-004): Códigos técnicos como HE-4330 y HE-4335 se confunden en espacios densos por tokenización sub-palabra. Comprobamos que el chunking por SKU ya resolvía la búsqueda densa en 8/8, pero implementamos un Retriever Híbrido (0.4 BM25 con regex de SKU + 0.6 FAISS con MMR) como defensa en profundidad, optimizando además el ranking en garantías (MSG-006).
- Paradoja de la Métrica en Cargas Peligrosas (MSG-007): El validador aprueba la respuesta por contener "24". Sin embargo, el PDF restringe a 24 unidades solo a Zonas 4 y 5; Santa Rosa es Zona 3 (solo lleva recargo de 15%). Demuestra que una métrica laxa en verde puede ocultar un fallo normativo del LLM.
- Descuento vs. Flete Bonificado (MSG-001): Ante una consulta por $1.200.000, el sistema reconoce el 5% de descuento por volumen pero advierte que el flete bonificado exige $1.500.000 netos, debiendo abonarse el envío ($68.000 a Villa María).
- Reglas Sutiles de Facturación (MSG-013 a MSG-016): Además de frenar por crédito y mora, la lógica determinística aplicó restricciones críticas que alteran los netos calculados: tope de 15% en herramientas Dewalt, Makita y Bosch (Art. 5) y el 3% financiero por pago anticipado (Art. 6).
- La Trampa del Interrupt: Evaluar grafos con interrupt() en modo batch muta el estado final a confirmar_pedido. Nuestro arnés audita el checkpoint en SQLite antes del resume para constatar que el sistema supo frenar a tiempo.

7. Conclusiones y Viabilidad de Puesta en Producción

El proyecto demuestra que el paradigma de **IA Engineering** supera con creces las arquitecturas basadas exclusivamente en prompts:

- **Eficiencia y Blindaje Comercial:** Automatiza con absoluta confiabilidad el 70% de las interacciones rutinarias, mientras protege la solvencia de la compañía canalizando el 30% restante hacia aprobación gerencial y soporte especializado.
- **Resiliencia y Trazabilidad:** La integración de LangGraph con *SqliteSaver* erradica el riesgo de transacciones huérfanas ante contingencias de red o caídas de servidor, manteniendo trazabilidad completa del ciclo de vida del pedido.
- **Calidad de Software:** La solución se encuentra completamente empaquetada, con 27 tests unitarios automatizados en verde (*pytest*), configuración centralizada libre de constantes mágicas y respaldo cuantitativo persistido en *artefactos/salidas/evaluacion.json*.